AGENTS · PART 17

Agents That Survive a Crash.

A 40-step agent that dies at step 37 and restarts from zero is not an agent. It is a bill. Durable execution is how you fix that.

`checkpoint` `resume` `idempotent` `replay`

16 slides · checkpoints · idempotency · approval gates · Temporal vs LangGraph

AI Coach John Follow

PROITBRIDGE Strive For Better Future

Repost

What "durable" actually means

A durable agent persists its progress outside the process. When the process dies, a new one picks up exactly where the old one stopped.

WITHOUT DURABILITY



36 LLM calls, 36 tool calls, every side effect: gone. Some of them ran twice.

WITH DURABILITY



Each step's result was written to a store. The new process reads the log and continues.

WHY AN AI ENGINEER MUST CARE

Money
A deep-research run can cost ≈$2–10 in tokens. Losing it on a 429 is a real line item at scale.

Side effects
Restart from zero means the email is sent twice and the card is charged twice.

Time
Agents that wait for a human can take days. No process lives that long.

Debugging
A persisted step log is the only way to answer "why did it do that?"

Six ways a long agent dies

The longer an agent runs, the more certain it is that one of these happens. A 40-step task with a 1% per-step failure rate fails ≈33% of the time.

20–50	1%	0
tool calls in a typical research or coding task	per-step failure → ≈33% task failure at 40 steps	steps recovered without a checkpoint store

01 **Rate limits.** One HTTP 429 from the model provider mid-run. Your retry logic covers the call, not the 30 steps before it.

02 **Timeouts.** Serverless functions cap at 15 min (Lambda) or 60 min (Cloud Run). Gateways cut idle HTTP at 30–60 s.

03 **Deploys and restarts.** Kubernetes reschedules the pod. Every in-memory agent loop on it vanishes.

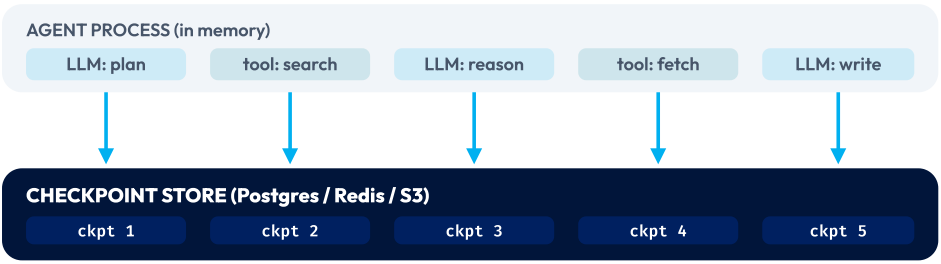
04 **Tool failures.** The search API is down for 5 minutes. The agent throws, the run is gone.

05 **Context overflow.** Step 30 pushes the history past the window. No state means no way to compact and continue.

06 **Waiting on a human.** "Approve this refund?" gets answered tomorrow. The process that asked is long dead.

Checkpoint after every step

Treat each LLM call and each tool call as a transaction. When it completes, write the new state to a store before doing anything else.



WHAT GOES INTO A CHECKPOINT

Conversation
Full message list including tool results, or a pointer to it plus a compacted summary.

Position
Which node or step is next. The plan, if the agent made one. Loop counters.

Pending action
A tool call that was issued but not confirmed. This is the dangerous gap.

Identity
thread_id, run_id, user, model version, prompt version. You will need these to replay.

Write the checkpoint **before** you return the result to the loop. Crash-after-write is recoverable. Crash-before-write is a repeat.

Resume from the last good state

Recovery is just: load the latest checkpoint for this thread, rebuild the in-memory state, call the next step. The loop does not know it died.

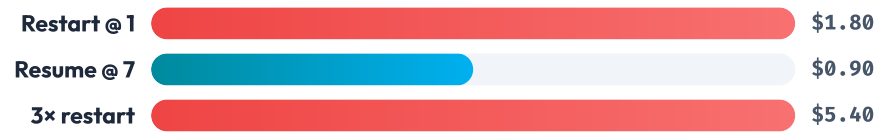


WHO TRIGGERS THE RESUME?

`queue redelivers the job` · `cron finds stale runs` · `user clicks "continue"` · `webhook`

The trigger must be outside the process that died. Usually a queue (SQS, Pub/Sub, Celery, BullMQ).

RESTART VS RESUME, 12-STEP RESEARCH RUN AT ≈$0.15 PER STEP



Lock the thread on resume. Two workers restoring the same checkpoint at once will both run step 7 and both send the email.

PATTERN 3

Idempotent tool calls and idempotency keys

Resume means a step may run twice. Idempotency means running it twice equals running it once.



TOOL CALL	NATURALLY IDEMPOTENT?	MAKE IT SAFE BY
get_weather(city)	Yes, a read	Nothing. Just cache the result in the checkpoint.
upsert_row(id, data)	Yes, keyed write	Use the run_id + step as the row key.
send_email(to, body)	**No**	Idempotency-Key header (Resend, SendGrid) or a dedupe table.
charge_card(amount)	**No**	Stripe idempotency_key = run_id:step. Same key, one charge.

Derive the key from deterministic state: **run_id + step_index + tool_name**. Never from a timestamp or a fresh uuid, or every retry gets a new key.

Exactly-once is a lie, mostly

You cannot guarantee a network call runs exactly once. You can guarantee it runs at least once and that repeats are harmless. Together that behaves like once.

GUARANTEE	WHAT CAN HAPPEN	AGENT EXAMPLE	USE WHEN
At-most-once	Step may be skipped	Fire-and-forget analytics ping	Loss is fine, duplicates are not
At-least-once	Step may repeat	Queue redelivers the job after a crash	Default for every durable system
Effectively-once	Repeats are absorbed	At-least-once + idempotency key	Payments, emails, DB writes

at-least-once delivery + **idempotent handler** = **effectively once**

The gap that bites: tool call succeeded, process died before the checkpoint was written. Resume re-issues it.

✕ **Wrong mental model**

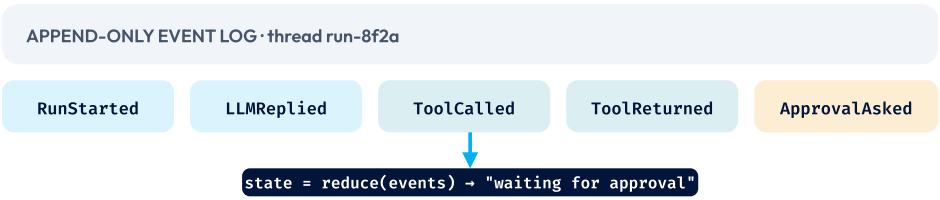
"My framework has retries, so I'm safe." Retries create the duplicate problem. They do not solve it.

✓ **Right mental model**

Every side-effecting tool gets a key and a dedupe check. Reads are free to repeat. Design tools that way from day one.

Event-sourced agent logs

Instead of overwriting one state blob, append immutable events. Current state is the fold of all events. You get history, audit and replay for free.




```
# one row per event, never UPDATE
CREATE TABLE agent_events (
  thread_id   text,
  seq         int,          -- monotonic per thread
  type        text,         -- LLMReplied | ToolCalled | ...
  payload     jsonb,
  created_at  timestamptz DEFAULT now(),
  PRIMARY KEY (thread_id, seq)  -- duplicate seq = rejected
);
```

+ You gain

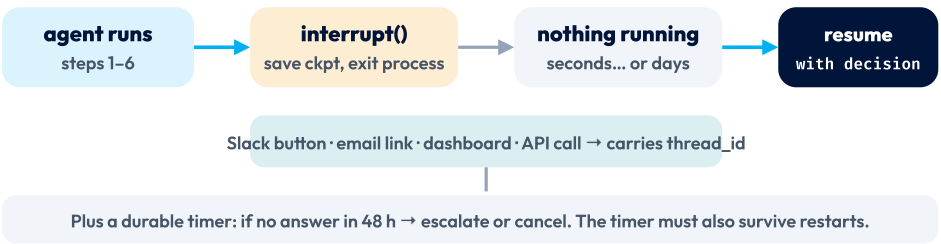
Audit trail per step, replay from any point, cheap "what did the model see" queries, no lost-update races.

− You pay

Replaying 500 events to rebuild state is slow. Snapshot every N events (LangGraph does this per super-step).

Approval gates that wait for days

"Confirm the $4,000 refund?" might be answered in 3 seconds or 3 days. The agent must pause without holding a process, a thread or an HTTP connection open.



FRAMEWORK	PRIMITIVE	HOW IT WAITS
LangGraph	`interrupt()` / `Command(resume=)`	Checkpointer persists state; graph exits; invoke again with same thread_id
Temporal	`workflow.wait_condition` + `signal`	Workflow is durable by design; signals delivered whenever they arrive
Inngest / Restate	`step.waitForEvent()`	Function suspends server-side; event resumes it, up to days later
OpenAI Agents SDK	`needs_approval` + `RunState`	Serialize RunState, store it, rebuild and continue after approval

Design the state schema first

Whatever you checkpoint has to be serialized, versioned and read back by code you have not written yet. Keep it small and boring.

```python
from typing import TypedDict, Annotated, Literal
from langgraph.graph.message import add_messages

class AgentState(TypedDict):
    schema_version: int                       # 3  → migrate old ckpts
    messages: Annotated[list, add_messages]   # append-only reducer
    plan: list[str]                           # what is left to do
    step: int                                 # next plan index
    artifacts: dict[str, str]                 # S3 keys, NOT file bytes
    pending_tool: dict | None                 # issued, unconfirmed call
    status: Literal["running", "waiting_human", "done", "failed"]
    model: str                                # "claude-sonnet-5" pinned
```

✓ **JSON-serializable only.** No open sockets, clients, DataFrames or lambdas in state.

✓ **Pointers, not payloads.** A 20 MB PDF goes to object storage; the state holds its key.

✓ **Version it.** A run paused Friday resumes Monday on new code. Migrate or reject old versions.

✓ **Reducers over overwrites.** Append messages; merge artifacts. Parallel branches then compose safely.

✗ **Don't** store secrets, raw API keys or PII you would not want in a 90-day retention table.

✗ **Don't** let state grow unbounded. Compact messages past ≈50 turns into a summary node.

Workflow engines vs framework checkpointers

Checkpointers save agent state. Workflow engines make the whole function durable: retries, timers and waits included.

OPTION	YOU GET	FIT
LangGraph + PostgresSaver	Per-step checkpoints, interrupt/resume, time travel. Queues and retries are yours.	Teams already on LangGraph
Temporal	Deterministic replay, retries, timers, signals, versioning. Used at Uber, Stripe, OpenAI.	Enterprise, many long workflows
Inngest	step.run / sleep / waitForEvent over HTTP. Serverless-friendly, hosted.	Next.js, Vercel, small teams
Restate	Journal-based replay, virtual objects, low latency, single binary.	Latency-sensitive services
AWS Step Functions	Managed JSON state machine, 1-year max run, native waits and Lambda calls.	All-in AWS shops
DIY queue + DB	No lock-in. You write locking, retries, timers and idempotency yourself.	Learning, very simple agents

TYPICAL PRODUCTION STACK

engine: retries · timers · human waits → **activity = agent loop + its own checkpoints**

HANDS ON

Durable LangGraph in 25 lines

Swap the in-memory saver for Postgres, add an interrupt before the risky tool, and resume by thread_id.

```python
from langgraph.checkpoint.postgres import PostgresSaver
from langgraph.types import interrupt, Command

def refund_node(state):
    ok = interrupt({"ask": f"Refund ${state['amount']}?"}) # pauses
    if not ok: return {"status": "cancelled"}
    stripe.Refund.create(amount=state["amount"],
        idempotency_key=f"{state['run_id']}:refund") # re-run safe
    return {"status": "done"}

with PostgresSaver.from_conn_string(DB_URL) as saver:
    saver.setup()
    app = graph.compile(checkpointer=saver)
    cfg = {"configurable": {"thread_id": "run-8f2a"}}

    app.invoke({"amount": 4000, "run_id": "run-8f2a"}, cfg)
    # → returns with __interrupt__; process can exit

    # hours later, new process, same thread_id:
    app.invoke(Command(resume=True), cfg) # resumes refund_node
```

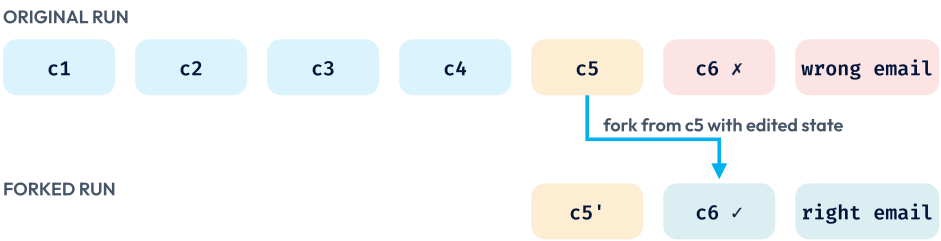
Every node boundary is a checkpoint. A crash inside a node re-runs that node only.

`app.get_state_history(cfg)` lists every checkpoint for time-travel debugging.



Replay and time-travel debugging

Once every step is persisted, "why did the agent email the wrong customer?" becomes a query, and "what if I fix the prompt at step 5?" becomes a fork.



Steps 1–4 are not re-executed. No tokens spent, no side effects repeated.

Inspect

Open checkpoint 5: exact messages, tool outputs and the model's raw reply. No guessing from logs.

Fork

Edit state (fix the retrieved doc, change the plan), resume from there. LangGraph: `update_state` + invoke.

Regression test

Saved checkpoints become fixtures. Replay 50 real failures against a new prompt before shipping it.

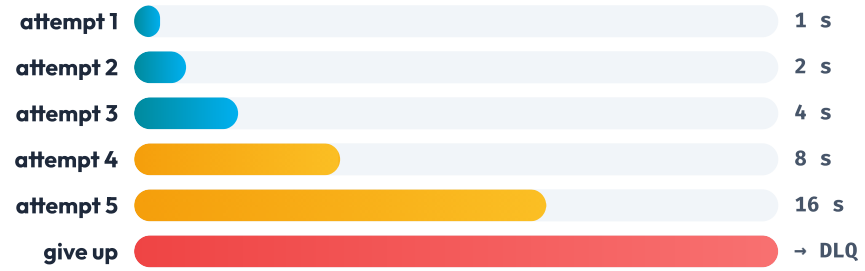
Determinism caveat

LLM output is not deterministic. Replay re-uses recorded outputs; Temporal enforces this, LangGraph does not.

Retries, backoff and the dead-letter queue

Durability decides what survives a failure. Retry policy decides how hard you try before giving up, and where the run goes when you do.

EXPONENTIAL BACKOFF WITH JITTER, 5 ATTEMPTS



ERROR	RETRY?	POLICY
429 rate limit	Yes	Honor Retry-After; back off; consider a fallback model
5xx / timeout	Yes	Exponential + jitter, max 5, idempotency key on the call
400 bad request	No	Prompt or schema bug; retrying burns money. Fail fast to DLQ
Model refuses / loops	Bounded	Max 2 re-prompts, then escalate to human

The dead-letter queue is not a graveyard. It is a work list: a human inspects the checkpoint, fixes state, and re-queues the run.

TRADE-OFFS

What durability costs you

Every checkpoint is a write. Every resume is a read plus deserialization. Small next to an LLM call, but not zero, and not always worth it.

≈5–20 ms
Postgres checkpoint write per step

≈1–3 s
a typical LLM step, so overhead is ≈1%

≈50–500 KB
state per step once history grows

✓ Worth it when

Runs exceed ≈1 minute or ≈10 steps · any side-effecting tool · human approval in the loop · you bill customers per run · you need an audit trail.

— Skip it when

Single LLM call with a retry · pure read-only agents under 10 s · prototypes and demos · throw-away batch jobs you can re-run wholesale.

HIDDEN COSTS TO BUDGET FOR

Storage growth. 10K runs × 40 steps × 200 KB ≈ 80 GB. Set a retention policy.

Schema migrations. Paused runs on old state versions must still resume.

PII in checkpoints. Messages contain user data. Encrypt at rest, redact, expire.

Ops. Temporal is a cluster to run. Managed clouds (Temporal Cloud, Inngest) trade money for that.

Durable agents on one slide

CONCEPT	REMEMBER
Why	429s, timeouts, deploys, tool outages, human waits. 1%/step ≈ 33% failure at 40 steps.
Checkpoint	Persist state after every LLM and tool step, before returning to the loop.
Resume	Load latest checkpoint by thread_id, lock, continue. Trigger from a queue.
Idempotency	Key = run_id:step:tool. At-least-once + idempotent handler = effectively once.
Event log	Append-only events, state = fold(events). Snapshot every N steps.
Human gate	interrupt() / signal / waitForEvent. Process exits; resume carries the answer.
State schema	JSON only, pointers not payloads, schema_version, compact past ≈50 turns.
Tooling	LangGraph PostgresSaver for state; Temporal / Inngest / Restate for durable code.
Replay	Inspect any checkpoint, fork with edited state, replay as regression fixtures.
Retries	Backoff + jitter on 429/5xx, fail fast on 400, DLQ is a human work list.

Persist every step. Key every side effect. Assume the process will die, and design so nobody notices.


AI Coach John Follow PROITBRIDGE Strive For Better Future Repost